Six-Max Preflop Equilibria via Monte Carlo CFR

James Wu

October 2026

Abstract

(Abstract to be written last.)

Contents

Notation **iii**

1 Introduction **1**
1.1 Motivation and scope . . . 1
1.2 A short introduction to six-max hold'em . . . 1
1.3 The game we solve . . . 3
1.4 Abstractions . . . 3
1.5 Related work . . . 4
1.6 Contributions and outline . . . 5

2 Mathematical Background **6**
2.1 Preliminaries . . . 6
2.1.1 Extensive-form games . . . 6
2.1.2 Strategies, best responses and equilibria . . . 6
2.1.3 Regret and online learning . . . 6
2.2 Counterfactual regret minimisation . . . 6
2.3 Monte Carlo CFR . . . 6
2.4 Weighted averaging and other variants . . . 6
2.5 More than two players . . . 6
2.6 Estimators used in this work . . . 6
2.6.1 Equity . . . 6
2.6.2 Realization-adjusted pot shares . . . 6
2.6.3 Self-normalised importance sampling . . . 6
2.7 Fitting a payoff model to observed play . . . 6

3 Methodology **7**
3.1 Encoding the game . . . 7
3.2 Equity computation . . . 8
3.3 The realization model . . . 8
3.4 The solver . . . 10
3.5 Two solves: Scratch and Anchored . . . 11
3.6 Reference charts and the distance metric . . . 11
3.7 Tuning by coordinate search . . . 11
3.8 A fixed-strategy fit . . . 12
3.9 The trainer . . . 12

4 Results and Discussion **16**
4.1 Solver correctness on a solvable game . . . 16
4.2 Convergence and performance of the six-max solve . . . 16
4.3 Agreement with the reference charts . . . 16
4.4 Where the model and the charts disagree . . . 16
4.5 The fixed-strategy fit . . . 16

4.6 Limitations 16
4.7 Conclusion and future work 16

Bibliography **17**

A Bet sizes and game parameters **19**

B Fitted realization parameters **20**

C Further range charts **21**

Notation

Symbol	Meaning
$N = \{1, \dots, n\}$, c	players; the chance player
H, Z	decision histories; terminal histories
$P(h)$, $A(h)$	player to act at h; legal actions at h
$\mathcal{I}_i$, I	player i's information partition; an information set
σ, σ^t, $\bar{\sigma}$	a behavioural strategy profile; the iterate at time t; the average strategy
$\pi^\sigma(h)$, $\pi^\sigma_i(h)$, $\pi^\sigma_{-i}(h)$	reach probability; player i's part; everyone else's (chance included)
u_i	utility of player i
$v^\sigma_i(I)$, $v^\sigma_i(I, a)$	counterfactual value of I; with a forced at I
$R^T(I, a)$, R^T_+	cumulative counterfactual regret; its positive part
$\varepsilon(\sigma)$	exploitability
$k \in \{0, \dots, 168\}$, $b(x)$, n_k	hand class; the class of combo x; combos in class k (6, 4 or 12)
eq_i	equity of player i
pot, C_i, rake(z)	pot; player i's contribution; rake taken at terminal z
ρ_i, θ	realization factor of player i; the realization parameters
S, γ, κ	stack-to-pot ratio at the flop; depth fade $\gamma = S/(S + \kappa)$; its half-life
τ	softmax temperature

Chapter 1

Introduction

1.1 Motivation and scope

Poker has long served as a benchmark for reasoning under imperfect information [1]. Unlike chess or Go, a player does not see the whole state of the game: the opponents' cards are hidden, so good play has to be randomised, and the value of an action depends on what the opponents believe as much as on the cards themselves. The last decade has seen a series of landmark results built on one family of algorithms, *counterfactual regret minimisation* (CFR) [25]. Heads-up limit hold'em was essentially solved by Cepheus [2]. DeepStack [17] and Libratus [4] beat professional players at heads-up no-limit hold'em, and Pluribus [6] did so in the six-player game.

These systems need large compute budgets and solve the game street by street, re-solving subgames as play unfolds. Human players study a far smaller object: *preflop charts*, tables of what to do with each starting hand before any community cards are dealt. Preflop is the decision every player faces in every hand, the stage where most hands end, and the one whose structure is simple enough to tabulate. Commercial tools publish such charts, computed by solving the full game on large clusters, but the charts come without the model behind them.

This report studies the preflop stage of six-player ('six-max') no-limit hold'em cash games at a stack depth of 100 big blinds as a finite game in its own right. We

(i) define a preflop-only game whose payoffs at the moment the flop would be dealt approximate the value of playing the rest of the hand, through a parametric *realization model*;

(ii) solve it with external-sampling Monte Carlo CFR [13] with linear averaging [5], on a laptop, in about a minute;

(iii) fit the realization model's parameters so that the solution matches published equilibrium charts [19], and study a cheaper fitting method that avoids a solve per candidate;

(iv) build an interactive trainer on top of the solution that grades a user's decisions by importance-sampled rollouts.

Everything is implemented in our own software, Preflop, written in C++20 with a raylib front end [21] and the PokerHandEvaluator library [14], with Python tooling for the fitting.

The equilibrium we compute is an equilibrium of *our* model, not of real poker. How close it gets to charts computed from the full game is therefore the central empirical question, and the realization model is where most of the modelling effort goes.

1.2 A short introduction to six-max hold'em

We assume no knowledge of poker; Sklansky [22] and Chen and Ankenman [7] are standard references for the game and its mathematics.

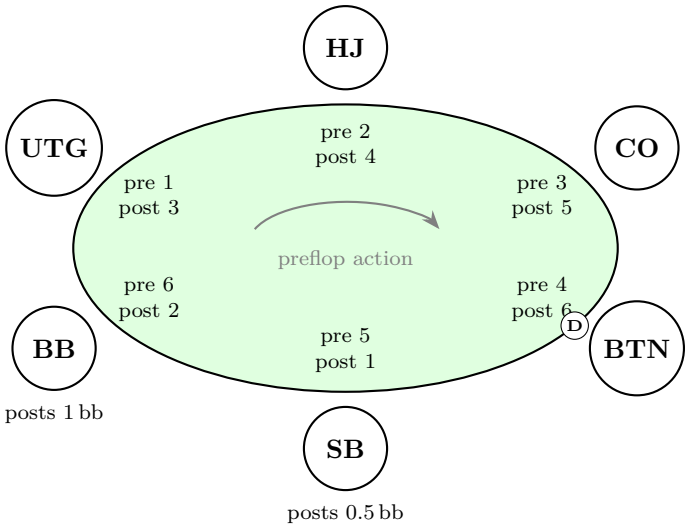


Figure 1.1: A six-max table. The numbers give each seat's order of action preflop ('pre') and on every later street ('post'). D marks the dealer button. Action moves clockwise.

Cards and hands. Each player is dealt two private *hole cards* from a standard 52-card deck. Up to five *community cards* are then dealt face up in three stages: the *flop* (three cards), the *turn* and the *river* (one card each). A player's hand is the best five-card combination of their two hole cards and the five community cards. Hands are ranked from high card, one pair and two pair, through three of a kind (a *set* when it uses a pocket pair), a straight (five consecutive ranks, the ace playing high or low, so A-2-3-4-5, the *wheel*, counts) and a flush (five cards of one suit), up to a full house, four of a kind and a straight flush.

Betting. There are four betting rounds: one before the flop (*preflop*) and one after each of the flop, turn and river. In each round a player facing a bet may *fold* (give up the hand and anything already put in), *call* (match the bet) or *raise* (increase it). In no-limit hold'em a raise may be any size up to the player's whole stack. A hand ends when all but one player have folded, who wins the pot uncontested, or at a showdown after the river, where the best hand wins.

Positions and blinds. At a six-player table the seats are named by their position relative to the *button* (Figure 1.1): under the gun (UTG, also called the lojack, LJ), hijack (HJ), cutoff (CO), button (BTN), small blind (SB) and big blind (BB). Before the cards are dealt, SB and BB post forced bets of 0.5 and 1 big blind (bb), which give the others something to win. Preflop, action starts with UTG and runs clockwise, so the blinds act last. On every later street the order changes: SB acts first and BTN last. Acting last postflop, being *in position* (IP) against the remaining players, is a large advantage, since that player sees what everyone else did before deciding. All amounts in this report are in big blinds, and all stacks start at 100 bb.

Preflop vocabulary. The first voluntary raise is an *open*; folding to it or calling it are the alternatives for the players behind. A re-raise of an open is a *3-bet*, a re-raise of that a *4-bet*, and so on (the blinds count as the first bet). Calling an open without having put money in yet is a *cold call* or *flat*; a 3-bet after an open and one or more calls is a *squeeze*. A 4-bet called by a 3-bettor, or a 5-bet all-in, typically commits the whole stack.

Starting hands. There are $\binom{52}{2} = 1326$ two-card *combos*. Before the flop the suits are interchangeable, so combos fall into 169 *hand classes*: 13 pocket pairs (6 combos each, e.g. AA), 78 suited non-pairs (4 combos, e.g. AKs) and 78 offsuit non-pairs (12 combos, e.g. AKo). We

draw them on the usual 13×13 grid: pairs on the diagonal, suited hands above it, offsuit hands below.

Equity and realization. A hand's *equity* against given opponent hands is its expected share of the pot if all remaining community cards were dealt and the hand went to showdown with no further betting. Equity is easy to compute but overstates what a hand actually wins: postflop betting forces folds, and a hand out of position, or one that rarely makes strong combinations, wins less than its equity. Players speak of how well a hand *realises* its equity. Suited and connected hands make flushes and straights and realise well, offsuit hands with a weak kicker realise badly, and position helps everyone. This is the idea our payoff model formalises (Sections 2.6.2 and 3.3).

1.3 The game we solve

Players and stakes. Six players with 100 bb each, blinds 0.5/1, stacks reset every hand. The game ends at the flop: whenever betting closes with two or more players left, we stop and pay out according to a model of the remaining play (Section 2.6.2).

Actions and sizes. Every decision is a subset of $\{F, C, R\}$ (fold, call, raise) with one raise size per level, chosen to match the reference charts [19]:

- opens are to 2.5 bb, or 3 bb from SB;
- 3-bets are 3× the open in position and 5× from the blinds, plus one open size per caller (a squeeze);
- 4-bets are 2× the 3-bet in position and 2.55× out of position;
- a 5-bet is all-in, and facing an all-in a player may only call or fold.

Raise totals are rounded to 0.1 bb. For example, BB 3-bets a BTN open to 12.5 bb, and BTN can 4-bet to 25 bb. Appendix A lists every size.

Cold calling. Players may call an open, and later players may overcall it, but once a 3-bet is in, players who have not yet put money in may only fold or raise. A player who cold-called the open may also call a squeeze. Limping (calling the big blind without raising) is not allowed.

Rake. As in the online games the reference charts are computed for, the house takes 5% of the pot, capped at 4 bb, but only when a flop is dealt or an all-in is called ('no flop, no drop'). The game is therefore negative-sum for the players.

Size. With these rules the betting tree has 7,499 decision nodes and 8,529 terminal histories: 1,703 where everyone else folds, 5,797 called all-ins and 1,029 that reach a flop, 378 of them with three or more players. The longest line has 19 actions. A fragment of the tree is shown in Figure 1.2. With one information set per decision node and hand class, a player has $7{,}499 \times 169 \approx 1.27 \times 10^6$ information sets in total across all seats.

1.4 Abstractions

Real no-limit hold'em is far too large to solve directly: even heads-up it has more than 10^{160} decision points [12]. Our game differs from the real game in three ways.

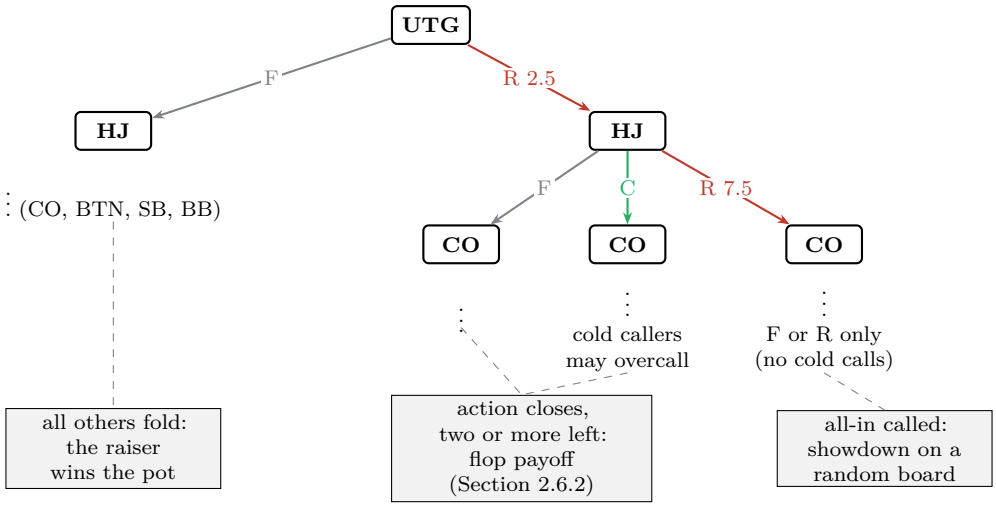


Figure 1.2: A fragment of the preflop betting tree from UTG's first decision. Each decision node is one history; every node is paired with the actor's hand class to give an information set. The three kinds of terminal are shown at the bottom.

Action abstraction. One raise size per level, no limping, and no cold calls of 3-bets. These match the reference charts' own rules, so they are restrictions shared with the target rather than errors relative to it. Abstraction can have counter-intuitive effects on the quality of an equilibrium [24], which is one reason we keep the rules identical to the charts'.

Card abstraction. Strategies depend on the hand class, not the combo. Preflop, this loses nothing: relabelling suits maps the game onto itself, so an equilibrium can be taken suit-symmetric [10]. The solver still deals real combos from one deck, so *card removal* (holding an ace makes the opponents' aces less likely) is modelled exactly.

Payoff abstraction. This is the large one. We do not play the flop, turn and river. Instead, a terminal where the flop would be dealt pays each live player a share of the pot given by their equity, scaled by a *realization factor* that depends on the hand, the player's role (position, caller or raiser) and the depth of the pot. The factors have a dozen or so parameters, which we fit to the charts (Sections 3.3 and 3.7).

1.5 Related work

CFR was introduced by Zinkevich et al. [25], and its Monte Carlo variants by Lanctot et al. [13]. CFR$^+$ [23] solved heads-up limit hold'em [2], and linear and discounted weighting schemes [5] speed up convergence further. Neller and Lanctot [18] give an accessible tutorial. For more than two players CFR has no equilibrium guarantee, but it still removes dominated actions [9] and produces strong play in practice: Pluribus [6] used Monte Carlo CFR with linear weighting for its six-player blueprint. Function approximation has also been combined with CFR [3, 17].

Preflop-only games have been studied in the form of *push/fold* (all-in or fold) games for short stacks, where the equilibrium can be computed exactly for two players and approximately for three [8]. Our game is much richer: deep stacks, four raise levels, cold calls and squeezes, with postflop value replaced by a model. Fitting payoff parameters so that observed play becomes rational is a case of inverse game theory, related to learning game parameters from equilibrium play [15] and to quantal-response models of behaviour [16].

1.6 Contributions and outline

The contributions of this report are:

- a precise definition of a six-player preflop game with rake and a realization-adjusted payoff model built from hand features (suitedness, connectivity including wheel straights, pairs, position, initiative, domination and pot depth);
- a fast external-sampling MCCFR solver with lazy linear weighting and lock-free threading, checked against an exactly solvable heads-up game;
- a fit of the model to published charts, a solve with the charts locked that exposes where model and charts disagree, and a study of a cheap fixed-strategy fitting method, including why it fails where it does;
- an interactive trainer that grades decisions with importance-sampled rollouts and standard errors.

Chapter 2 develops the mathematics from first principles: extensive-form games and equilibria, regret, CFR and its Monte Carlo variants, the estimators we use, and the statistics of fitting. Chapter 3 describes the model and its implementation: the encoding of the game, equity, the realization features, the solver, the reference charts, both fitting methods and the trainer. Chapter 4 reports correctness checks, convergence, agreement with the charts, where the model and the charts disagree, and the outcome of the fitting study. The appendices list the bet sizes, the fitted parameters and further range charts.

Chapter 2

Mathematical Background

2.1 Preliminaries

2.1.1 Extensive-form games

2.1.2 Strategies, best responses and equilibria

2.1.3 Regret and online learning

2.2 Counterfactual regret minimisation

2.3 Monte Carlo CFR

2.4 Weighted averaging and other variants

2.5 More than two players

2.6 Estimators used in this work

2.6.1 Equity

2.6.2 Realization-adjusted pot shares

2.6.3 Self-normalised importance sampling

2.7 Fitting a payoff model to observed play

Chapter 3

Methodology

This chapter turns the mathematics of Chapter 2 into a working system. Figure 3.1 shows the pipeline: an exact heads-up equity table is computed once; a solve from scratch produces the *Scratch* strategy; its realization parameters are fitted to reference charts; a second solve with the charts locked produces the *Anchored* strategy; and the trainer plays both.

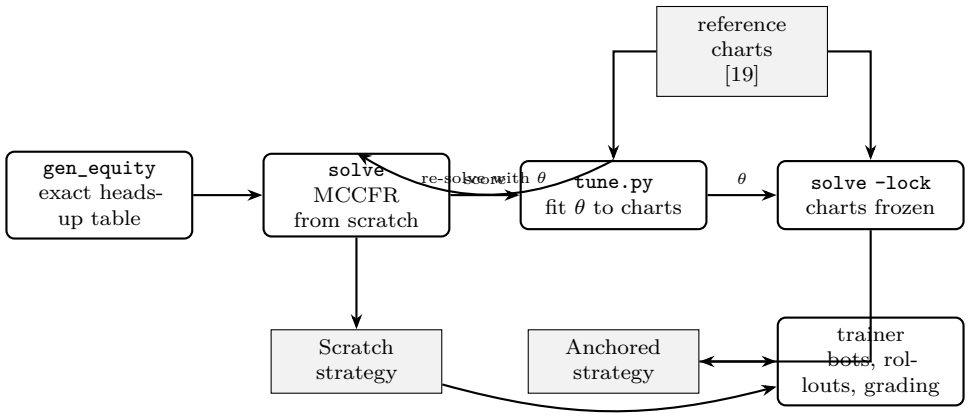


Figure 3.1: The pipeline that produces the two shipped strategies. The tuning loop re-solves from scratch for every candidate θ.

3.1 Encoding the game

Rules and tree. All rules of Section 1.3 live in one configuration structure (stack, blinds, raise multipliers, tick, rake, and whether a cold caller may call a squeeze), and a rules engine applies them to a hand state. The betting tree is built once by walking the engine from the empty history: each decision node stores the actor, a mask of legal actions and a child index per action, with -1 marking a terminal. Because all stacks are equal, an all-in is always a plain showdown with no side pots.

Terminals. Everything a terminal's payoff depends on, apart from the cards, is fixed by its history: each player's contribution C_i, the pot after rake, the stack-to-pot ratio S at the flop, which players are live, who acts last postflop, who made the last raise, whether the pot was 3-bet or more, and whether an all-in was called. These are precomputed once per terminal edge, so the solver traverses the tree by node index and never touches the rules engine in its inner loop.

Information sets and deals. An information set is a pair (decision node, hand class), and strategies and regrets are stored per pair (Section 1.4). Each iteration deals six real combos from

one shuffled deck, so blockers shape the opponents' hands exactly, while the strategy lookup uses only the class. Smaller games are solved on the same tree by forcing the early seats to fold at the root: `-seats 2` gives SB versus BB, which we use as a correctness check in Section 4.1.

3.2 Equity computation

Heads-up: an exact table. For two disjoint combos the equity is a sum over all $\binom{48}{5} = 1{,}712{,}304$ boards (Section 2.6.1). We precompute it for every ordered pair of disjoint combos. Matchups that differ only by a relabelling of suits, or by swapping the two sides, have equal (or complementary) equity, so each canonical matchup is evaluated once and the result is expanded into a 1326×1326 table of 32-bit floats (6.7 MB). Lookups are then exact and free. One simplification is that the table ignores the cards of folded players; we return to this in Section 4.6.

Multiway: sampled boards. With three or more players we draw $K = 100$ boards from the deck minus all twelve hole cards and average each player's share of the pot (Section 2.6.1). This is the solver's hot loop, so the seven-card evaluator is specialised. Each card is given a 64-bit key that packs a base-5 digit for its rank and a 4-bit count for its suit. A seven-card hand's key is then the sum of its cards' keys, and a board's key is computed once and shared by every player. Hands without a flush are ranked by a table lookup on the two rank fields, built at start-up from PokerHandEvaluator [14]; the rare flushes fall back to the library itself. The results are bit-identical to calling the library directly.

All-ins. A called all-in uses a single sampled board and pays the actual pot share on it. This estimator is unbiased for the expected payoff, and the noise averages out over iterations.

3.3 The realization model

At a terminal where the flop would be dealt, player i receives the share $s_i = \rho_i \mathrm{eq}_i / \sum_j \rho_j \mathrm{eq}_j$ of the pot after rake (**??**). This section defines the realization factor $\rho_i > 0$. It is built from features of the hand, the player's *role* and the depth of the pot. Fold equity and blockers are *not* modelled here: the solver deals real cards and plays the preflop betting out exactly, so both are already accounted for, and a blocker term would count them twice.

Depth. With S the stack behind divided by the pot at the flop, and a parameter $\kappa \geq 0$, let

$$\gamma = \frac{S}{S + \kappa} \in (0, 1]. \tag{3.1}$$

γ is close to 1 in deep single-raised pots and falls in 3-bet and 4-bet pots; at $S = \kappa$ it equals $\frac{1}{2}$, so κ reads as the 'half-life' SPR (Figure 3.2). Terms whose value comes from playing future streets are scaled by γ, so they fade when little money is left behind.

Roles. Call player i an *out-of-position caller* if i neither acts last postflop among the live players nor made the last raise, so has neither position nor initiative, and an *in-position caller* if i acts last but did not make the last raise. Call the pot *re-raised* ($\mathbb{1}_{\mathrm{rr}} = 1$) if it was 3-bet or more; this is read from the betting history, not from S. The position factor is

$$\mathrm{pos}_i = \begin{cases} \alpha_{\mathrm{ip}}\, \chi_i & i \text{ acts last postflop,} \\ 1 - (1 - \alpha_{\mathrm{oop}})\, \gamma_i' & \text{otherwise,} \end{cases} \tag{3.2}$$

where $\chi_i = \chi$ for an in-position caller in a re-raised pot and 1 otherwise, and $\gamma_i' = \xi\gamma + (1 - \xi)$ for an out-of-position caller and γ otherwise. The out-of-position penalty $1 - \alpha_{\mathrm{oop}}$ shrinks with

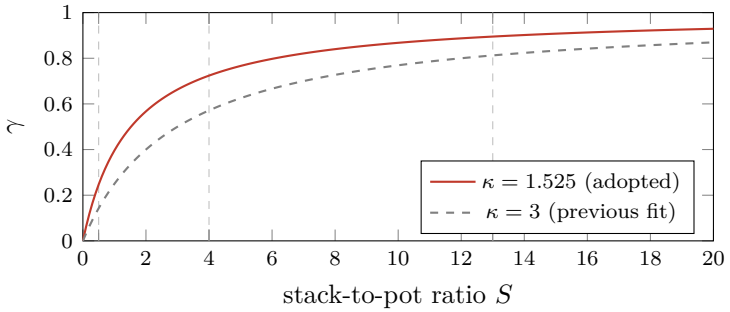


Figure 3.2: The depth fade $\gamma(S)$. Dashed grid lines mark typical flop SPRs: about 13 in a single-raised pot, 4 in a 3-bet pot and below 1 in a 4-bet pot.

depth, since with little behind there is little to be outplayed on; $\xi \in [0, 1]$ controls how much of that shrink an out-of-position caller gets. χ captures that calling a 3-bet in position, with a capped range into a small SPR against the aggressor's initiative, is worth less than position in a single-raised pot.

Hand features. For a hand with ranks $r_1 \geq r_2$ (2 is 0, A is 12):

- $\mathrm{st}(r_1, r_2)$, the *connectivity*: the number of five-card straights, from the wheel A-2-3-4-5 (the ace playing low) up to T-J-Q-K-A, that contain both ranks; 0 for pairs. For example $\mathrm{st} = 4$ for 98, 2 for KQ, 1 for AK and A5, and 0 for K4.
- $\mathrm{dom}(r_1, r_2)$, the *domination*: for offsuit non-pairs with $r_1 \geq \mathrm{T}$, $\min(r_1 - 1 - r_2, 4)$, else 0. It measures how often the hand makes top pair with a weak kicker against better versions of itself: KQo has 0, KJo 1, AJo 2, and K6o is capped at 4.
- Indicators $\mathbb{1}_{\mathrm{suited}}$, $\mathbb{1}_{\mathrm{pair}}$, $\mathbb{1}_{\mathrm{off}}$ (offsuit non-pair) and $\mathbb{1}_{\mathrm{raiser}}$ (i made the last raise).

The hand factor is

$$H_i = 1 + \left(\phi\,\mathbb{1}_{\mathrm{suited}} + \eta\,\mathrm{st}(r_1, r_2)\,\gamma + \varsigma\,\mathbb{1}_{\mathrm{pair}}\,\gamma\right)\psi_i - \left(\delta + \delta_{\mathrm{rr}}\mathbb{1}_{\mathrm{rr}}\right)\mathrm{dom}(r_1, r_2)\,(1 - \mathbb{1}_{\mathrm{raiser}}) \\ - \lambda\,(12 - r_1)\,\mathbb{1}_{\mathrm{off}} - \beta\,(12 - r_1)\,\mathbb{1}_{\mathrm{pair}}, \tag{3.3}$$

with $\psi_i = \psi$ for an out-of-position caller and 1 otherwise, and finally

$$\rho_i = \max\left(\mathrm{pos}_i \cdot H_i \cdot (1 + \iota\,\mathbb{1}_{\mathrm{raiser}}),\ 0.05\right). \tag{3.4}$$

The floor keeps every factor positive, so the denominator of the share never vanishes.

Why this shape. The structure is chosen so that the solver produces *polarised* re-raising ranges, as real solvers do. Hands that realise badly when calling, such as dominated offsuit broadways, prefer to raise or fold. Hands whose value is postflop playability, such as suited aces, suited connectors and small pairs, get that value in deep pots, which mostly means calling. Suited wheel aces like A5s keep their straight and flush terms even as the aggressor, which makes them natural bluffs. An earlier model, with ρ a function of position and hand class only and shrunk towards 1 in shallow pots, barely depended on role and produced linear 3-betting ranges: small pairs looked like good 3-bets and A5s lost its edge. Figure 3.3 shows the resulting factors for four roles.

realization heat-grids (generated)

Figure 3.3: The realization factor ρ for every hand class under the adopted parameters, in four roles. Each grid is the 13×13 class grid: pairs on the diagonal, suited hands above it, offsuit below.

Symbol	Code name	Role and rationale
α_{oop}	`oop`	position factor out of position ($\alpha_{\text{ip}} = 1$ fixes the scale)
ϕ	`flush`	suited hands make flushes: a bonus at any depth
η	`conn`	per straight using both cards, scaled by γ: straights pay off only with money behind
ς	`set`	pairs make sets ('set mining'), scaled by γ
δ	`dom`	per point of domination, for callers only: a dominated caller meets the hands that crush it
δ_{rr}	`domPot`	extra domination penalty in re-raised pots, where the raiser's range is narrower
λ	`low`	offsuit non-pairs, per rank the top card sits below an ace
β	`pairLow`	pairs, per rank below aces: overcards come on most flops
κ	`spr`	half-life SPR of the depth fade γ
ι	`init`	bonus for the last aggressor (initiative)
ψ	`oopCall`	scales the playability bonuses of out-of-position callers
ξ	`callerShrink`	how much of the depth shrink of the position penalty an out-of-position caller gets
χ	`ipCall`	scales position for in-position callers in re-raised pots

Table 3.1: The realization parameters θ. Fitted values are in Appendix B.

Neutral setting. With $\phi = \eta = \varsigma = \delta = \delta_{\text{rr}} = \lambda = \beta = \iota = 0$, $\kappa = 0$ and $\psi = \xi = \chi = 1$, every $H_i = 1$ and $\rho_i \in \{1, \alpha_{\text{oop}}\}$: a position-only model. We use it as a baseline in Section 4.3.

3.4 The solver

The solver implements external-sampling MCCFR (**??**) with linear averaging applied a block at a time (**??**). The traverser cycles through the seats, the deal and every non-traverser action are sampled from their true probabilities, and the traverser enumerates its actions.

Data layout. Regrets and strategy sums are flat arrays of single-precision floats of shape [nodes][169][3], two tables of about 15 MB each. The tree is a few levels deep, so the traversal is a plain recursion over node indices, and each terminal's payoff inputs are read from the precomputed table of Section 3.1.

Lazy linear weighting. Linear CFR multiplies both tables by $k/(k+1)$ after every block of $N = 10{,}000$ iterations, which gives block j a weight proportional to j. Rather than sweep the tables, we add each increment of block j multiplied by j. The tables then hold the discounted sums times a common factor, and regret matching and the normalised average are both invariant to such a factor (**??**), so the strategies are identical up to rounding and no pass over the tables is ever needed.

Threads. Iterations run in parallel in the style of Hogwild! [20]: all threads share the two tables and update them with relaxed atomic additions, without locks. Each thread has its own random number generator, seeded from the solver's. Because the lazy weights need no synchronised discount pass, threads never wait for each other. With one thread a run is bit-reproducible for a fixed seed; with more it is not, which is why our comparisons always use several seeds.

Average strategy. The output strategy is the strategy sum normalised per information set. Some information sets are reached only through their owner's own enumeration, after an action that player never takes, and accumulate no strategy sum; there we output the current regret-matched strategy, which is uniform only if the set was never reached. This matters for the trainer, whose bots must respond sensibly when the user deviates.

Checkpoints. The full state (configuration, iteration count, random state and both tables) is written after every chunk of iterations, to a temporary file that is then renamed, so a crash never leaves a half-written file. A checkpoint made under different rules is refused. One made under different realization parameters is accepted as a *warm start*: the regrets are kept and the strategy sums and iteration count restart.

3.5 Two solves: Scratch and Anchored

Scratch is MCCFR from scratch over the whole tree with the fitted parameters, 3×10^8 iterations. It is the strategy we compare with the reference charts.

Anchored freezes the reference charts. At every (spot, class) pair a chart covers, the solver plays the chart's probabilities, both when sampling and when the actor is the traverser, and never updates the strategy sum, so the output keeps the chart exactly. Everything else, notably the squeezes, cold 4-bets and overcalls that the charts do not cover, is solved as usual and so becomes a best response to players who follow the charts. Anchored is warm-started from the Scratch checkpoint. Otherwise, after an action a chart never takes (a call in a 3-bet-or-fold chart, say), the players behind reach that spot only when the locked player deviates, which happens only while the locked player is the traverser, so they would never accumulate regret there and would stay uniform.

The lock report. Regrets at locked entries are still accumulated, though never played. At the end of an Anchored solve, the mean regret of the best action against the chart's mix measures how much the chart gives up, in our model, against everything else. Summed over a spot's classes and ranked, these 'lock reports' point to where the model and the charts disagree most (Section 4.4).

3.6 Reference charts and the distance metric

Source. Our reference is the publicly available range viewer of PokerTrainer.se [19], which publishes GTO charts, and a second 'PTO' set adapted to typical opponents, for six-max 100 bb cash games with rake. Both sets cover the same 65 spots: 5 opens, 15 responses to an open, 15 responses to a 3-bet, 15 to a 4-bet and 15 to a 5-bet all-in. The charts are mixed: each cell gives a raise and a call frequency. We read them with a small browser-automation script and store them in a plain-text format with one file per spot. The bet sizes of our game are those of the charts (Appendix A), so differences are not sizing artefacts. All chart grids reproduced in this report are credited to this source.

Distance. We score a solved strategy $\bar{\sigma}$ against a chart at spot s with the weighted total-variation distance of **??**: each class k is weighted by its combo count n_k times the probability π_k that our strategy brings the class to the spot through the actor's own earlier actions, so classes we never bring there do not count. A spot's distance lies in $[0, 1]$, with 0 meaning identical play. Spots are grouped into the five categories above; the overall score averages the spots within each category, then averages the categories, so the fifteen deep 4-bet and 5-bet spots do not outweigh the five opens.

3.7 Tuning by coordinate search

The parameters θ are chosen to minimise the score plus a penalty for the UTG and BTN open frequencies leaving a band of ± 2.5 percentage points around the charts' own open widths. The

search is a coordinate pattern search in the style of Hooke and Jeeves [11] (Section 2.7):

- each round evaluates $\theta \pm \delta_j e_j$ for every coordinate j, each candidate being a *fresh* full solve averaged over a few seeds, the same seeds for every candidate;
- every coordinate whose better step beats the current point by more than z standard errors moves, all together, falling back to the best single move if the combination is worse; the standard error comes from the run-to-run spread pooled over all candidates of the round;
- when nothing moves the steps halve and the iteration count may grow;
- at the end, the winner and the start point are re-scored at the final iteration count.

An earlier version evaluated candidates by short warm-started runs from a converged solve. That ranked candidates by how fast the solve moved away from the old solution rather than where it would end up, and it stalled; fresh solves are slower but measure the right thing. One round over thirteen parameters costs about 26 solves per seed, roughly ten minutes at 5×10^7 iterations. This cost, a full solve per candidate, is what motivates the cheaper method of the next section.

3.8 A fixed-strategy fit

3.9 The trainer

The trainer deals the user a spot, plays the five other seats as bots, and grades the user's decision. It has a headless core, which is unit-tested, and a graphical front end written with raylib [21].

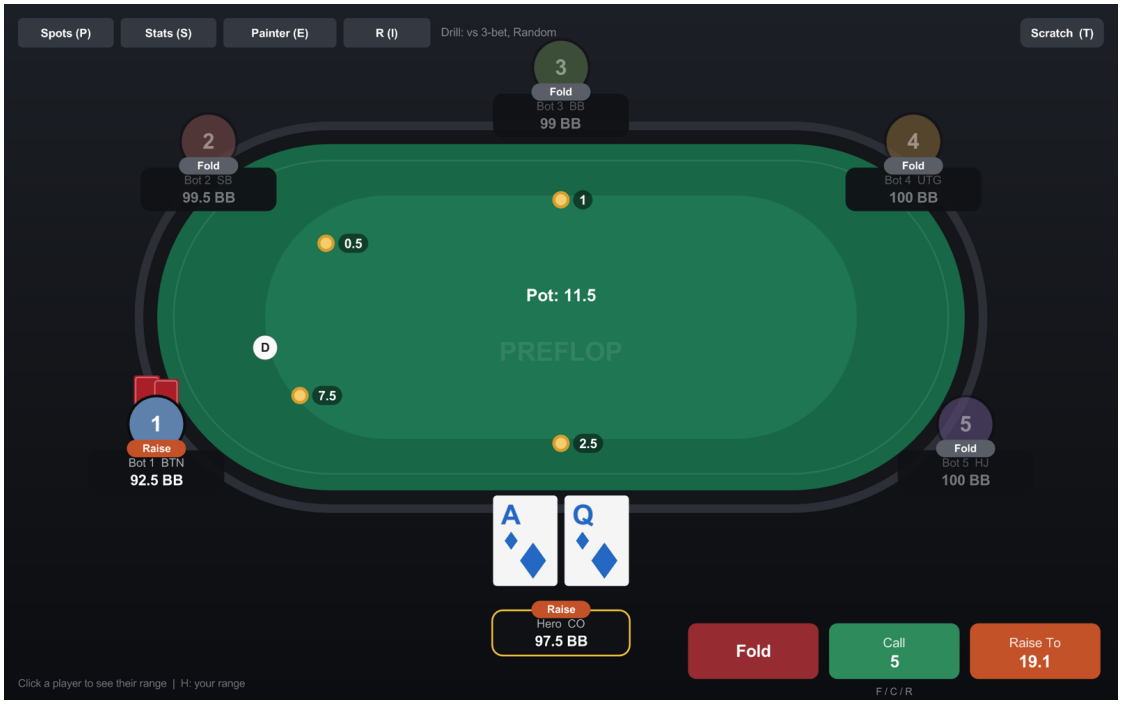
Spots and bots. A spot of a requested type (open, facing an open, squeeze, facing a 3-bet, facing a squeeze, facing a 4-bet or more) and position is found by rejection sampling: deal a hand, let the bots and the user's own earlier decisions play from the strategy, and accept it when the user reaches a matching decision. Bots sample their actions from the strategy's average, so they stay mixed and respond sensibly to off-equilibrium play by the user.

Rollout EVs. Reading action values from sampled regrets would be far too noisy. Instead, for the user's actual hand, the trainer re-deals the opponents' cards from the deck minus the user's, weights each deal by the probability that the bots would have produced the observed history with those cards, and plays every legal action out on the same deals with everyone following the strategy. This is the self-normalised importance-sampling estimator of **??**. Using the bots' actual cards would leak hidden information. The trainer keeps dealing until the effective sample size reaches 2000 (or a cap of two million deals), and it reports a standard error per action and per difference between actions; since all actions share the deals, differences are far more precise than levels.

Grading. A choice is correct if its EV loss against the best action is at most 0.1 bb, or if the strategy plays it at least 10% of the time and loses at most 1 bb, or if the loss is within two standard errors. In the chart modes below, a decision covered by a chart is graded by the chart alone: correct if and only if the chart plays the action at least 10% of the time.

Modes. A key cycles three strategies: *Scratch*; *GTO*, which is Anchored with the GTO charts exactly where charted; and *PTO*, the PTO charts written over Anchored. Statistics (accuracy, average EV loss and the most-leaked spots) are kept per mode.

Interface. Figure 3.4 shows a hand from the trainer. The user, in the cutoff with A♢Q♢, opened and faces a 3-bet from the button; the buttons offer a fold, a call of 5 bb or a 4-bet to 19.1 bb. After the hand, the feedback panel shows the EV of each action with its standard error, the grade, the strategy's mix for the hand and the range for the whole spot. Other screens include a spot picker, statistics, a painter for editing charts and a realization inspector (Figure 3.5), which shows ρ for every class in a chosen role, pot type and SPR.



(a) The decision: CO holds AQs against the button's 3-bet to 7.5 bb.



(b) Feedback at the end of the hand. CO 4-bet (the strategy's most frequent action) and the button moved all-in; the panel shows CO's range facing the all-in, AQs's mix (fold 99.8%), and rollout EVs: calling loses 6.3 bb more than folding.

Figure 3.4: The trainer's table view and feedback panel, Scratch mode. Cell colours follow the action (raise, call, fold); classes the strategy never brings to the spot are greyed out.

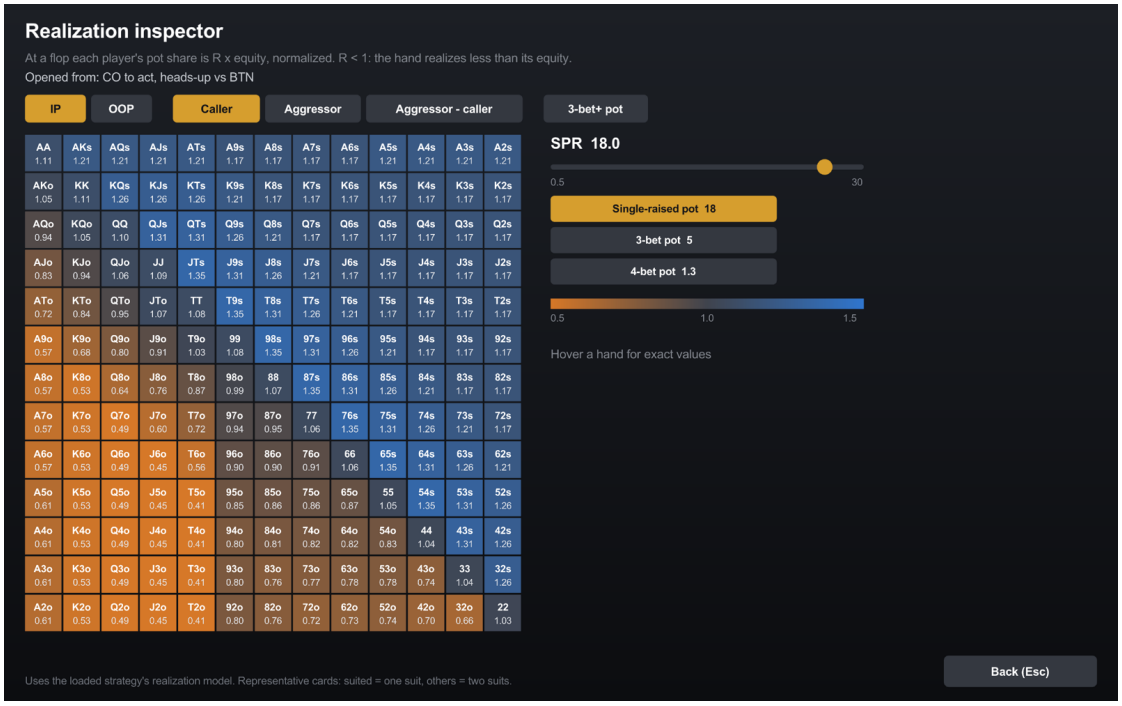


Figure 3.5: The realization inspector, showing ρ for an in-position caller in a single-raised pot at SPR 18. Suited connectors reach 1.35; dominated offsuit hands such as A9o fall to 0.57.

Chapter 4

Results and Discussion

4.1 Solver correctness on a solvable game

4.2 Convergence and performance of the six-max solve

4.3 Agreement with the reference charts

4.4 Where the model and the charts disagree

4.5 The fixed-strategy fit

4.6 Limitations

4.7 Conclusion and future work

Bibliography

[1] D. Billings, A. Davidson, J. Schaeffer and D. Szafron. 'The Challenge of Poker'. In: *Artificial Intelligence* 134.1–2 (2002), pp. 201–240. DOI: 10.1016/S0004-3702(01)00130-8.

[2] M. Bowling, N. Burch, M. Johanson and O. Tammelin. 'Heads-up Limit Hold'em Poker is Solved'. In: *Science* 347.6218 (2015), pp. 145–149. DOI: 10.1126/science.1259433.

[3] N. Brown, A. Lerer, S. Gross and T. Sandholm. 'Deep Counterfactual Regret Minimization'. In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. Vol. 97. Proceedings of Machine Learning Research. 2019.

[4] N. Brown and T. Sandholm. 'Superhuman AI for Heads-Up No-Limit Poker: Libratus Beats Top Professionals'. In: *Science* 359.6374 (2018), pp. 418–424. DOI: 10.1126/science.aao1733.

[5] N. Brown and T. Sandholm. 'Solving Imperfect-Information Games via Discounted Regret Minimization'. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 33. 2019. DOI: 10.1609/aaai.v33i01.33011829.

[6] N. Brown and T. Sandholm. 'Superhuman AI for Multiplayer Poker'. In: *Science* 365.6456 (2019), pp. 885–890. DOI: 10.1126/science.aay2400.

[7] B. Chen and J. Ankenman. *The Mathematics of Poker*. ConJelCo, 2006.

[8] S. Ganzfried and T. Sandholm. 'Computing an Approximate Jam/Fold Equilibrium for 3-Player No-Limit Texas Hold'em Tournaments'. In: *Proceedings of the 7th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2008.

[9] R. Gibson. 'Regret Minimization in Games and the Development of Champion Multiplayer Computer Poker-Playing Agents'. PhD thesis. University of Alberta, 2014.

[10] A. Gilpin and T. Sandholm. 'Lossless Abstraction of Imperfect Information Games'. In: *Journal of the ACM* 54.5 (2007). DOI: 10.1145/1284320.1284324.

[11] R. Hooke and T. A. Jeeves. '"Direct Search" Solution of Numerical and Statistical Problems'. In: *Journal of the ACM* 8.2 (1961), pp. 212–229. DOI: 10.1145/321062.321069.

[12] M. Johanson. *Measuring the Size of Large No-Limit Poker Games*. 2013. arXiv: 1302.7008.

[13] M. Lanctot, K. Waugh, M. Zinkevich and M. Bowling. 'Monte Carlo Sampling for Regret Minimization in Extensive Games'. In: *Advances in Neural Information Processing Systems 22 (NIPS)*. 2009.

[14] H. Lee. *PokerHandEvaluator*. Version 0.6.0. URL: https://github.com/HenryRLee/PokerHandEvaluator.

[15] C. K. Ling, F. Fang and J. Z. Kolter. 'What Game Are We Playing? End-to-end Learning in Normal and Extensive Form Games'. In: *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)*. 2018, pp. 396–402. DOI: 10.24963/ijcai.2018/55.

[16] R. D. McKelvey and T. R. Palfrey. 'Quantal Response Equilibria for Normal Form Games'. In: *Games and Economic Behavior* 10.1 (1995), pp. 6–38. DOI: 10.1006/game.1995.1023.

[17] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson and M. Bowling. 'DeepStack: Expert-Level Artificial Intelligence in Heads-Up No-Limit Poker'. In: *Science* 356.6337 (2017), pp. 508–513. DOI: 10.1126/science.aam6960.

[18] T. W. Neller and M. Lanctot. *An Introduction to Counterfactual Regret Minimization*. Model AI Assignments, Symposium on Educational Advances in Artificial Intelligence (EAAI). 2013.

[19] PokerTrainer.se. *Range Viewer: 6-max Cash, 100bb (GTO and PTO charts)*. URL: https://app.pokertrainer.se/range-viewer (visited on 18/10/2026).

[20] B. Recht, C. Ré, S. Wright and F. Niu. 'Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent'. In: *Advances in Neural Information Processing Systems 24 (NIPS)*. 2011.

[21] R. Santamaria. *raylib*. URL: https://www.raylib.com.

[22] D. Sklansky. *The Theory of Poker*. Two Plus Two Publishing, 1999.

[23] O. Tammelin. *Solving Large Imperfect Information Games Using CFR+*. 2014. arXiv: 1407.5042.

[24] K. Waugh, D. Schnizlein, M. Bowling and D. Szafron. 'Abstraction Pathologies in Extensive Games'. In: *Proceedings of the 8th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*. 2009.

[25] M. Zinkevich, M. Johanson, M. Bowling and C. Piccione. 'Regret Minimization in Games with Incomplete Information'. In: *Advances in Neural Information Processing Systems 20 (NIPS)*. 2007.

Appendix A

Bet sizes and game parameters

All amounts are in big blinds. Raise totals round to the nearest 0.1 bb (halves away from zero) and are capped at the stack. The sizes are those of the reference charts [19]. A single out-of-position 4-bet multiplier cannot reproduce both of the charts' sizes of 19 and 23 after rounding; 2.55 gives 19.1 and 23.

Action	Rule	Examples
Open	2.5; 3 from SB	UTG opens to 2.5; SB opens to 3
3-bet in position	3× the open	BTN vs a CO open: 7.5; BB vs an SB open: 9
3-bet from the blinds	5× the open	BB vs a BTN open: 12.5
Squeeze	as a 3-bet, +1 open per caller	BTN vs HJ open, CO call: 10; BB vs CO open, two calls: 17.5
4-bet in position	2× the 3-bet	BTN vs BB's 12.5: 25
4-bet out of position	2.55× the 3-bet	CO vs BTN's 7.5: 19.1; SB vs BB's 9: 23
5-bet	all-in	100
Stacks, blinds	100; 0.5/1	reset every hand
Rake	5% of the pot, capped at 4	only if a flop is dealt or an all-in is called
Limping	not allowed	
Cold calls	of an open only	a cold caller may also call a squeeze

Table A.1: Bet sizes and rules of the game.

Appendix B

Fitted realization parameters

Table B.1 lists the realization parameters (Table 3.1) at three points: the values found by the coordinate search of Section 3.7, the optimum of the fixed-strategy fit at temperature $\tau = 1$ (Section 3.8), and the adopted values, halfway between the two (Section 4.5). The scale is fixed by $\alpha_{\mathrm{ip}} = 1$.

Symbol	Code name	Coordinate search	Fixed-strategy fit	Adopted
α_{oop}	`oop`	0.7	0.712	0.706
ϕ	`flush`	0.16	$0.173,7$	$0.166,9$
η	`conn`	0.05	$0.050,8$	$0.050,4$
ς	`set`	0.1	$0.146,2$	$0.123,1$
δ	`dom`	0.1	$0.117,4$	$0.108,7$
λ	`low`	0.03	$0.049,0$	$0.039,5$
κ	`spr`	3	0.05	1.525
ι	`init`	0.2	$0.235,5$	$0.217,8$
β	`pairLow`	0.01	$0.004,2$	$0.007,1$
ψ	`oopCall`	1	1.045	$1.022,5$
ξ	`callerShrink`	0	0	0
χ	`ipCall`	0.8	0.869	$0.834,5$
δ_{rr}	`domPot`	0	$0.066,5$	$0.033,2$

Table B.1: Realization parameters. The fixed-strategy fit drove κ to its lower bound 0.05, which switches the depth fade off.

Appendix C

Further range charts

(Generated grids to be inserted.)